

Multiple Linear Regression

Sourav Karmakar

souravkarmakar29@gmail.com

OUTLINE

- Multiple Linear Regression: Hypothesis Function
- Cost Function
- Gradient Descent Algorithm
- Stopping criteria for Gradient Descent
- How to check if Gradient Descent is Working
- Implementation Note
 - Choosing the value of learning rate
 - Feature Scaling

Multiple Linear Regression : Hypothesis

Problem Statement:

Suppose there are k many features or predictor variables $x_1, x_2, x_3, \dots, x_k$ and a target variable y . Now we want to express y as a linear combination of the those predictor variables. We have a vector of features/predictors $\vec{x} = [x_1, x_2, x_3, \dots, x_k]^T$

Model Representation:

Here, we can express our multiple linear regression problem by means of following probabilistic model.

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \dots + \theta_k x_k + \varepsilon$$

Here we have a vector of model parameters $\Theta = [\theta_0, \theta_1, \theta_2, \dots, \theta_k]^T$ of dimension $k + 1$ and ε is the random error.

Like in case of simple linear regression, here also we shall use our sample data to estimate model parameter vector Θ i.e.: $\hat{\Theta}$.

Multiple Linear Regression : Hypothesis

Hypothesis Function:

The predicted value of the target variable is obtained from the estimated model parameter as:

$$\hat{y} = \hat{\theta}_0 + \hat{\theta}_1 x_1 + \hat{\theta}_2 x_2 + \cdots + \hat{\theta}_k x_k = \hat{\theta}_0 + \sum_{j=1}^k \hat{\theta}_j x_j$$

Where $\hat{\theta}_0, \hat{\theta}_1, \hat{\theta}_2, \dots, \hat{\theta}_k$ are estimated model parameters from the data.

Multiple Linear Regression : Cost Function

Cost Function:

Here, we shall define cost function in the following way:

$$\text{As, } \hat{y}^{(i)} = \hat{\theta}_0 + \sum_{j=1}^k (\hat{\theta}_j x_j^{(i)})$$

$$J(\hat{\Theta}) = \frac{1}{2m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2 = \frac{1}{2m} \sum_{i=1}^m \left(y^{(i)} - \hat{\theta}_0 - \sum_{j=1}^k (\hat{\theta}_j x_j^{(i)}) \right)^2$$

(Here $\langle \vec{x}^{(i)}, y^{(i)} \rangle$ is the i^{th} data sample and $\hat{y}^{(i)}$ is the corresponding output of the model)

Our Objective:

To estimate the model parameters $\hat{\Theta} = [\hat{\theta}_0, \hat{\theta}_1, \hat{\theta}_2, \dots, \hat{\theta}_k]^T$ from the given sample of data so that the cost function $J(\hat{\Theta})$ is minimized.

Multiple Linear Regression : Optimization

Gradient Descent algorithm for Cost Function optimization:

- **Gradient Descent** algorithm is an optimization technique for minimizing the cost function $J(\hat{\Theta})$
- **Gradient Descent** is an iterative algorithm and in a nutshell it does the following:
 - Start with some initial values of $\hat{\Theta} = [\hat{\theta}_0, \hat{\theta}_1, \hat{\theta}_2, \dots, \hat{\theta}_k]^T$
 - Keep Changing $\hat{\theta}_0, \hat{\theta}_1, \hat{\theta}_2, \dots, \hat{\theta}_k$ to reduce $J(\hat{\Theta})$ until we end up at a minimum.

Gradient Descent Algorithm

- **Algorithm:** Gradient Descent

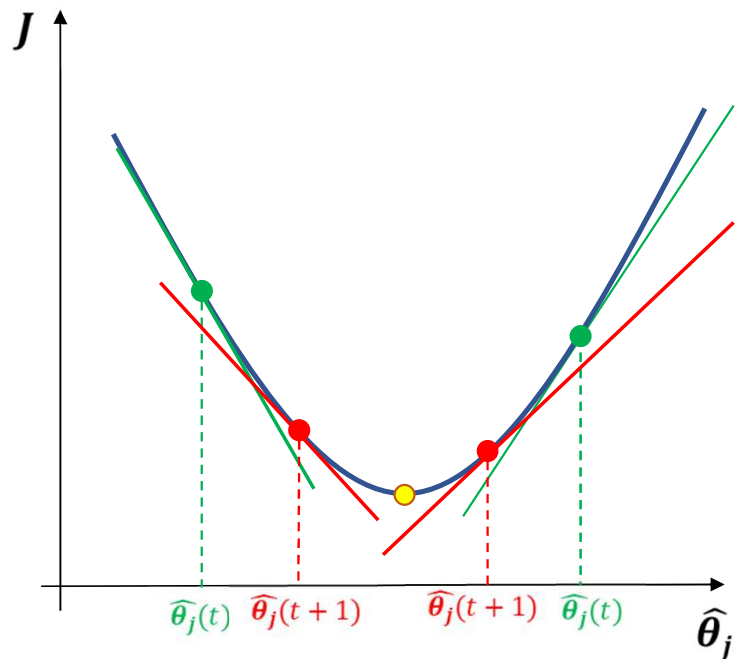
- **Input:** Feature set ($X = [\vec{x}^{(i)}]_{i=1}^m$) and corresponding values of target variable ($Y = [y^{(i)}]_{i=1}^m$).
- **Output:** Model parameters $\hat{\Theta} = [\hat{\theta}_0, \hat{\theta}_1, \hat{\theta}_2, \dots, \hat{\theta}_k]^T$ for which the cost function $J(\hat{\Theta})$ is minimum.
- **Initialisation:** Initialise the model parameters $\hat{\Theta} = [\hat{\theta}_0, \hat{\theta}_1, \hat{\theta}_2, \dots, \hat{\theta}_k]^T$ with random values.
iteration number: $t \leftarrow 1$
- Repeat until convergence {
 - Compute the cost function $J(\hat{\Theta})$
 - $\hat{\theta}_j(t+1) = \hat{\theta}_j(t) - \alpha \frac{\partial}{\partial \hat{\theta}_j} (J(\hat{\Theta}))$, simultaneously update for every $j = 0, 1, 2, \dots, k$
 - $t \leftarrow t + 1$}

*α is a small positive user defined parameter called the **Learning Rate***

Gradient Descent Algorithm : Intuition

Intuition of Gradient Descent algorithm:

- The plot of cost function with any of the parameters (considering the other parameters fixed) is of the following form.



- Now as per gradient descent rule:

$$\widehat{\theta}_j(t+1) = \widehat{\theta}_j(t) - \alpha \frac{\partial}{\partial \widehat{\theta}_j} (J(\widehat{\Theta}))$$

- Now if the gradient $\frac{\partial J}{\partial \widehat{\theta}_j}$ is *positive*, then the update in the value of $\widehat{\theta}_j$ which is calculated as $\widehat{\theta}_j(t+1) - \widehat{\theta}_j(t)$ is *negative*. Indicating that, in the next update $\widehat{\theta}_j$ would be smaller than its previous value.
- On the other hand if the gradient $\frac{\partial J}{\partial \widehat{\theta}_j}$ is *negative*, then the update in the value of $\widehat{\theta}_j$ is *positive*. Indicating that, in the next update $\widehat{\theta}_j$ would be greater than its previous value.

Gradient of Cost Function

- The Cost Function for Linear Regression with multiple predictor variable is:

$$J(\widehat{\Theta}) = \frac{1}{2m} \sum_{i=1}^m (y^{(i)} - \widehat{y}^{(i)})^2 = \frac{1}{2m} \sum_{i=1}^m \left(y^{(i)} - \widehat{\theta}_0 - \sum_{j=1}^k (\widehat{\theta}_j x_j^{(i)}) \right)^2$$

- We can calculate the derivative of the cost function with respect to $\widehat{\theta}_0$ and $\widehat{\theta}_j$ (*for* $j = 1, 2, 3, \dots, k$) as following:

$$\frac{\partial}{\partial \widehat{\theta}_0} (J(\widehat{\Theta})) = \frac{1}{m} \sum_{i=1}^m \left(y^{(i)} - \widehat{\theta}_0 - \sum_{j=1}^k (\widehat{\theta}_j x_j^{(i)}) \right) (-1) = \frac{1}{m} \sum_{i=1}^m \left(\widehat{\theta}_0 + \sum_{j=1}^k (\widehat{\theta}_j x_j^{(i)}) - y^{(i)} \right)$$

$$\frac{\partial}{\partial \widehat{\theta}_j} (J(\widehat{\Theta})) = \frac{1}{m} \sum_{i=1}^m \left(y^{(i)} - \widehat{\theta}_0 - \sum_{j=1}^k (\widehat{\theta}_j x_j^{(i)}) \right) (-x_j^{(i)}) = \frac{1}{m} \sum_{i=1}^m \left(\widehat{\theta}_0 + \sum_{j=1}^k (\widehat{\theta}_j x_j^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

for $j = 1, 2, 3, \dots, k$

Gradient Descent update rule

- Hence, the update rule in Gradient descent algorithm would be (for t^{th} iteration):

$$\widehat{\theta}_0(t+1) := \widehat{\theta}_0(t) - \frac{\alpha}{m} \sum_{i=1}^m \left(\widehat{\theta}_0(t) + \sum_{j=1}^k \left(\widehat{\theta}_j(t) x_j^{(i)} \right) - \widehat{y}^{(i)}(t) \right) y^{(i)}$$

$$\text{and} \quad \widehat{\theta}_j(t+1) := \widehat{\theta}_j(t) - \frac{\alpha}{m} \sum_{i=1}^m \left(\widehat{\theta}_0(t) + \sum_{j=1}^k \left(\widehat{\theta}_j(t) x_j^{(i)} \right) - \widehat{y}^{(i)}(t) \right) x_j^{(i)}$$

for $j = 1, 2, 3, \dots, k$

- **Note:**

If we consider another predictor x_0 whose value is always 1 (i.e. $x_0^{(i)} = 1, \forall i$), then we can use the last equation as a generalized gradient descent update rule.

Gradient Descent update rule

- Training Data Set can be written as following:

Sl. No.	x_0	x_1	$\dots$	x_k	y
1	1	$x_1^{(1)}$	$\dots$	$x_k^{(1)}$	$y^{(1)}$
2	1	$x_1^{(2)}$	$\dots$	$x_k^{(2)}$	$y^{(2)}$
$\vdots$	$\vdots$	$\vdots$	$\dots$	$\vdots$	$\vdots$
m	1	$x_1^{(m)}$	$\dots$	$x_k^{(m)}$	$y^{(m)}$

- There are total k many features and m many training examples.
- Notice that we have added one extra feature column x_0 with all values 1 in the left.

- Hence, the Gradient Descent update rule for the parameters of linear regression (for t^{th} iteration) is:

$$\widehat{\theta}_j(t+1) := \widehat{\theta}_j(t) - \frac{\alpha}{m} \sum_{i=1}^m (\widehat{y}^{(i)}(t) - y^{(i)}) x_j^{(i)}$$

for $j = 0, 1, 2, 3, \dots, k$ and $x_0^{(i)} = 1, \forall i \in [m]$

Gradient Descent : Stopping Criteria

- **Stopping criteria for gradient descent:**

We can consider following two methods as stopping criteria for the gradient descent algorithm.

1. Convergence of the model parameters:

If in two successive iterations the change in the model parameters is *not significant* then we can say that the algorithm has converged. As the model parameters are vectors, to compare the values of the model parameters in two successive iterations we can either take norm or absolute differences. Alternatively if at least one of the parameters is not converged then we keep on running the algorithm until all the parameters are converged.

2. Predefined number of iterations:

We run the algorithm for predefined number of iterations.

Gradient Descent : Stopping Criteria

How do we know that the change in the model parameters is *not significant*

Suppose, $\widehat{\Theta}(t)$ is the estimated model parameter vector at t^{th} iteration of gradient descent and $\widehat{\Theta}(t + 1)$ is the estimated model parameter vector at $(t + 1)^{th}$ iteration of gradient descent.

Change in model parameter vector in two successive iterations:

$$\Delta\widehat{\Theta}(t) = \widehat{\Theta}(t + 1) - \widehat{\Theta}(t) = [\Delta\widehat{\theta}_0, \Delta\widehat{\theta}_1, \Delta\widehat{\theta}_2, \dots, \Delta\widehat{\theta}_k]$$

Now to check convergence we can use either of the following methods:

Method-1: Check whether L_2 norm of the change in model parameter vector is less than tolerance or not.

$$\|\Delta\widehat{\Theta}(t)\|_2 = \sqrt{\Delta\widehat{\theta}_0^2 + \Delta\widehat{\theta}_1^2 + \Delta\widehat{\theta}_2^2 + \dots + \Delta\widehat{\theta}_k^2} < tolerance$$

Method-2: Check the absolute value of change of individual model parameters is less than tolerance or not.

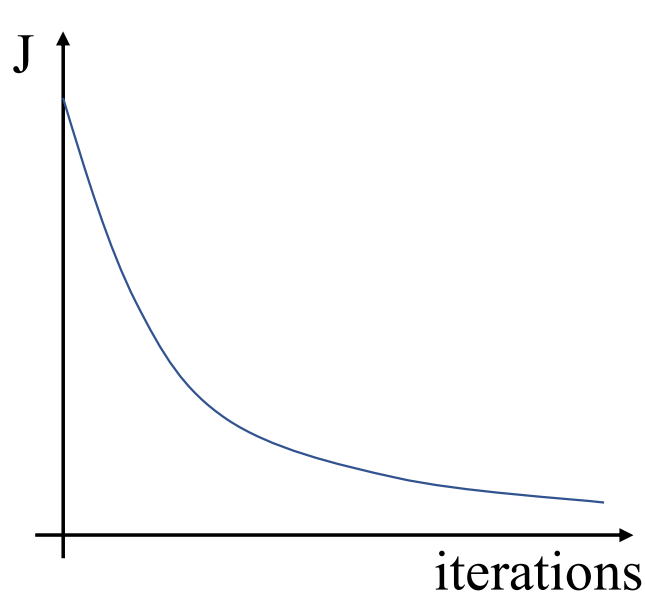
$$|\Delta\widehat{\theta}_0| < tolerance, |\Delta\widehat{\theta}_1| < tolerance, \dots, |\Delta\widehat{\theta}_k| < tolerance$$

tolerance is user defined small positive value (like: 10^{-4} or 10^{-6})

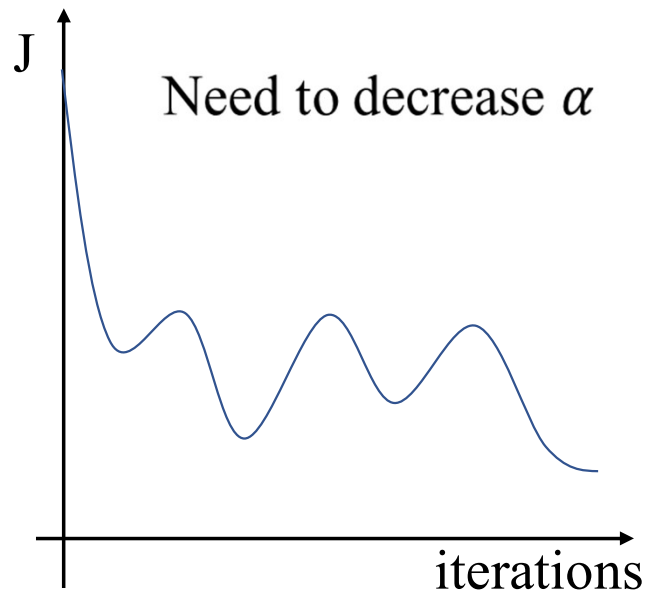
Gradient Descent : Troubleshooting

- **How to check whether gradient descent is working or not:**

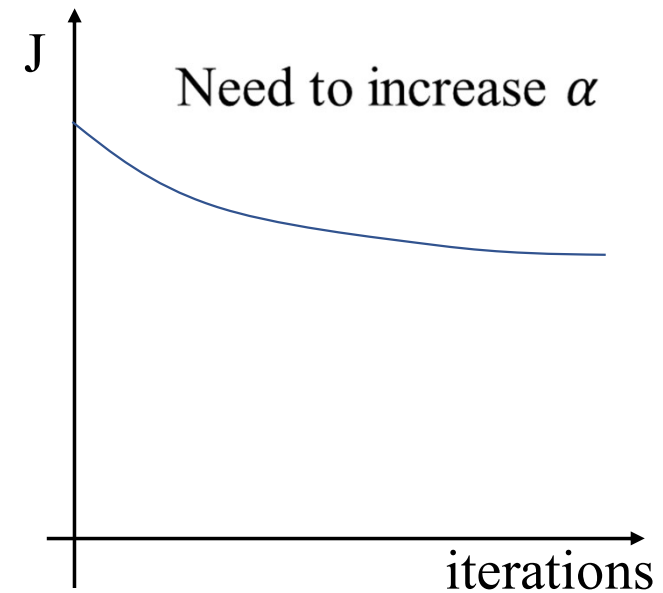
We can plot cost function versus number of iterations. If it's continuously decreasing then we can say that gradient descent is working. However we may have to fine-tune the value of “learning rate” (α) depending upon how fast or slow the cost function is converging.



Gradient Descent working properly



Gradient Descent not working properly



Gradient Descent : Implementation Note

- **Implementation Note-1: Choosing the value of Learning Rate α**

Higher value of learning rate α , may cause Gradient Descent algorithm to oscillate around the minimum point. Too low value of α will make Gradient Descent algorithm sluggish (i.e. too slow to converge). Hence, choosing a right value of α is important. There is no general *apply-to-all* value of learning rate.

As a rule of thumb to choose α people tries different values like:

..., 0.001, 0.003, 0.01, 0.03, 0.1, ...

Again this is not a exhaustive list of values of learning rate. One may choose different values like 0.005 as learning rate α .

Gradient Descent : Implementation Note

- **Implementation Note-2: Feature Scaling:**

Linear Regression and most of the other Machine Learning algorithms performs well when the range of the predictor variables are roughly in the same range. This is because, if the range of one predictor variable is significantly higher than another predictor variable, then the first one starts to dominate over the second one, making Gradient Descent algorithm sluggish. Hence, Feature Scaling is necessary for faster convergence of Gradient Descent algorithm. This done by either of the following two methods.

1. Standardization: Consider a predictor variable/ feature x_j whose mean is $\bar{x}_j$ and standard deviation is σ_{x_j} , then for feature scaling replace x_j with $\frac{x_j - \bar{x}_j}{\sigma_{x_j}}$, i. e. $x_j := \frac{x_j - \bar{x}_j}{\sigma_{x_j}}$,

This ensures that after scaling x_j will have zero mean and unit standard deviation.

2. Min-Max Scaling: $x_j := \frac{x_j - x_j^{\min}}{x_j^{\max} - x_j^{\min}}$, This ensures that after scaling x_j will be between 0 to 1

Thank You